

Time Complexities of Common Algorithms

Algorithm	Best Case	Average Case	Worst Case
Hashing (Search/Insert/Delete in Hash Table)	$O(1)$	$O(1)$	$O(n)$ (with collisions in worst-case chaining)
Binary Search (Sorted Array)	$O(1)$ (when found first attempt)	$O(\log n)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ (when always picking worst pivot)
Recursion (General case)	Depends on recurrence relation	Depends on problem	Depends on problem
Matrix Multiplication (Naive Approach)	$O(n^3)$	$O(n^3)$	$O(n^3)$
Strassen's Matrix Multiplication	$O(n^{2.81})$	$O(n^{2.81})$	$O(n^{2.81})$
Graphs - BFS (Breadth-First Search)	$O(V + E)$	$O(V + E)$	$O(V + E)$
Graphs - DFS (Depth-First Search)	$O(V + E)$	$O(V + E)$	$O(V + E)$
Backtracking	Depends on problem	Depends on problem	$O(2^n)$ (Exponential for worst case like N-Queens, Subset sum, etc.)
Dynamic Programming (Memoization/Tabulation)	Depends on subproblem size	Depends on problem	$O(2^n) \rightarrow O(n)$ (Optimized DP)

Detailed Explanations

1. Hashing (Hash Tables - HashMap in Java)

- **Best/Average Case: $O(1)$** (Ideal case when hash function distributes elements evenly)
- **Worst Case: $O(n)$** (When all elements collide into a single bucket)

2. Binary Search

- Works only on **sorted arrays** and **reduces search space by half each time**.
- **Time Complexity: $O(\log n)$** in all cases except best-case **$O(1)$** when the element is found immediately.

3. Merge Sort

- **Divide and Conquer algorithm** that **divides** the array into two halves, **sorts** them, and **merges** them back.
- **Time Complexity: $O(n \log n)$** in all cases (due to even splitting).

4. Quick Sort

- **Divide and Conquer algorithm** that **chooses a pivot** and partitions the array around it.
- **Best/Average Case: $O(n \log n)$**
- **Worst Case: $O(n^2)$** when pivot selection is bad (e.g., always choosing the smallest/largest element).

5. Recursion

- Time complexity **depends on the recurrence relation**.
- Example: Fibonacci (naive) **$O(2^n)$** , Factorial **$O(n)$** , Tower of Hanoi **$O(2^n)$** .

6. Matrix Multiplication

- **Naive Approach: $O(n^3)$**
- **Strassen's Algorithm: $O(n^{2.81})$** (Better but still costly)

7. Graph Algorithms

- **BFS (Breadth-First Search): $O(V + E)$** (Traverses all vertices and edges)
- **DFS (Depth-First Search): $O(V + E)$** (Traverses all vertices and edges)

8. Backtracking

- **Worst Case: $O(2^n)$** (N-Queens, Subset Sum, Sudoku Solver, etc.)
- **Best/Average:** Highly problem-dependent.

9. Dynamic Programming

- Converts **exponential recursive problems** into **polynomial time**.
- Example: Fibonacci (Recursive **$O(2^n)$** → DP **$O(n)$**), Knapsack, Longest Common Subsequence.

Summary of Key Algorithms

Algorithm	Best Case	Worst Case
Hashing	$O(1)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n^2)$
Recursion	Varies	Varies
Matrix Multiplication	$O(n^3)$	$O(n^3)$
Strassen's Algorithm	$O(n^{2.81})$	$O(n^{2.81})$
BFS/DFS	$O(V + E)$	$O(V + E)$
Backtracking	Varies	$O(2^n)$
Dynamic Programming	Varies	$O(2^n) \rightarrow O(n)$ (Optimized)